*Need help? Self-Service || Yammer || **No new spaces from Nov-23rd***

Android Dev Guide - Create InputHelper

Created by Akhil Malik, last modified on 18 Feb 2021

Creating New Input Helper

All new InputHelper should extend **InputHelperImpl** class. This is an abstract class which has helper attributes which need to be overridden and defined.

Once an InputHelper is created use setupInputHelper(helper: InputHelper) method to set input helper. In order to communicate between InputHelper and TextField, use textFieldInteractor.

Overriding Values

Every InputHelper has to define below mentioned values for it to work. You must define these values in order to use your input helper otherwise it will be an abstract class and can't be instantiated.

Example(Australian Mobile Number)-

Input Length - This value is used for any input character length on TextField.

```
override val inputLength: Int?
    get() = 10
```

Keyboard Type - This value is used for initialising a specific keyboard for TextField. You need to assign any of the InputType to this.

```
override val keyboardType: Int
    get() = InputType.TYPE_CLASS_PHONE
```

Allowed CharSet - This value is used to define charset that is allowed in TextField.

```
override val allowedCharSet: String?
    get() = InputHelper.ALLOWED_CHARS_NUMBER_ONLY
```

Delimiters - This value is used to define if there is any delimiter for formatting TextField value.

```
override val delimiter: String?
    get() = InputHelper.DELIMITER_SPACE
```

Delimiter Positions - This value is used to define position of delimiters on entered value. This field will be an array of positions.

```
override val label: Int
    get() = R.string.mobile_number
```

Default Label - This value is used to define default label for TextField.

```
override val label: Int
    get() = R.string.mobile_number
```

Default HelpText - This value is used to define default help text for TextField.

```
override val helpText: Int?
    get() = null
```

AutoFill - This value is used to define autofill for TextField. Use HintConstants to define.

```
override val autoFillType: String?
    get() = HintConstants.AUTOFILL_HINT_PHONE_NATIONAL
```

Value validation - This value is used to define value validation regex for TextField.

```
override val valueRegex: String?
    get() = InputHelper.VALUE_VALIDATOR_AUSTRALIA_MOBILE_NUMBER
```

valueError - This value is used to define value error. This error will be set to TextField if value validation fails.

```
override val valueError: Int?
    get() = R.string.error_textfield_mobile
```

chunkWhileTyping - This value is used to enable/disable chunking while typing for input value.

```
override val chunkWhileTyping: Boolean
    get() = false
```

Methods to override

Value validation (fun validateValue(): Boolean) -

In case we can't validate value with a simple regex, in that case we can override this method and validate value. Things to remember-

1. This method must return true if value validated otherwise false.
2. If validation fails error should be set by method itself and cleared if value validates.

Refer **InputHelperABN** for example implementation.

```
override fun validateValue(): Boolean {
    var status = true
    status = validateABN(textFieldInteractor.inputValue)
    if (!status) {
        textFieldInteractor.setErrorResource(R.string.error_textfield_abn)
    } else {
        textFieldInteractor.setErrorResource(null)
    }
    return status
}

private fun validateABN(abn: String): Boolean {
    var abn = abn
    val weights = intArrayOf(10, 1, 3, 5, 7, 9, 11, 13, 15, 17, 19)
    abn = abn.replace("\\D".toRegex(), "")
    if (abn.length == 11) {
        var sum = 0
        for (i in abn.indices) {
            val digit = abn[i] - '0' - if (i == 0) 1 else 0
            sum += weights[i] * digit
        }
        return sum % 89 == 0
    }
    return false
}
```

Value formatting (fun formatInput(charLength: Int?, delimiters: Array<Int>?)) -

In case we have different formatting rules at different length of input override this method.

Refer **InputHelperBankAccountNumber** for example implementation.

```
override fun formatInput(charLength: Int?, delimiters: Array<Int>?) {

    when (textFieldInteractor.inputValue.length) {
        6, 7 -> {
            super.formatInput(textFieldInteractor.inputValue.length, arrayOf(3))
        }
        8 -> {
            super.formatInput(textFieldInteractor.inputValue.length, arrayOf(4))
        }
        9, 10 -> {
            super.formatInput(textFieldInteractor.inputValue.length, arrayOf(3, 6))
        }
    }
}
```

Get delimiters (fun getDelimiters(): Array<Int>?) -

In case we have different delimiters for different scenario. This method can be used to specify those scenarios and return delimiter positions.

Refer **InputHelperCardNumber** for example implementation.

```
override fun getDelimiters(): Array<Int>? {
    return when {
        "^3[47][0-9]*$".toRegex().matches(textFieldInteractor.inputValue) -> {
            arrayOf(4, 10)
        } //AMEX
        "^3(?:0[0-5]|[689][0-9])[0-9]*$".toRegex().matches(textFieldInteractor.inputValue) -> {
            arrayOf(4, 10)
        }//DINERS
        "^4[0-9]*$".toRegex().matches(textFieldInteractor.inputValue) -> {
            arrayOf(4, 8, 12)
        }// VISA
        "^5[1-5][0-9]*|222[1-9][0-9]*|22[3-9][0-9]*|2[3-6][0-9]*|27[01][0-9]*|2720[0-9]*$".toRegex()
            .matches(textFieldInteractor.inputValue) -> {
            arrayOf(4, 8, 12)
        }// MASTER
        else -> {
            arrayOf(4, 8, 12)
        }
    }
}
```

No labels

Atlassian